

ORBIT DETERMINATION VIA PHYSICS INFORMED NEURAL NETWORKS

Andrea Scorsoglio*, Roberto Furfaro[†]

In this paper, a new method for solving orbit determination problems is introduced named Physics Informed Least Squares (PILS). We use a particular kind of single layer feed-forward neural networks called Extreme Learning Machines to enable higher accuracy and flexibility than classical least squares. The least squares estimate is used as baseline for the loss function, to which a regularizing term based on the differential equations modelling the dynamics of the problem is then added. This ensure that the learned relationship between input and output is compliant with the physics of the problem. The results are comparable or better than the batch least squares solution, with the advantage of not requiring an initial guess and being able to solve for the complete trajectory without needing any integration.

INTRODUCTION

The problem of orbit determination (OD) has always been of pivotal importance since the dawn of space flight. The success of virtually any mission is related to the possibility of estimating the state of the spacecraft at any given time. For instance, it is important to estimate the state of a spacecraft as soon as it leaves the launcher to assess the steps to take during the commissioning phase. The availability of reliable and accurate OD algorithms is also pivotal for planning correction maneuvers and station-keeping tasks, as well as for missions involving relative motion like rendezvous and docking. Historically, the task of orbit determination and state estimation has been solved via least squares estimation and Kalman filters and their respective derivations.

In the common least squares solution, a set of heterogeneous measurements are used to estimate a set of constant weights that multiplied by a predefined set of basis functions, provide a linear estimate of the measure.^{1,2} This has been extended to allow for non-linear relationships between parameters and measures: the resulting algorithm is normally referred to as non-linear least squares or sequential least squares. In this case, the problem cannot be solved with a simple matrix inversion but an iterative procedure must be implemented as the equations are not explicit in the parameters. Kalman filter KF³ and its variants⁴ are instead a class of recursive state estimator methods that apply to linear systems. The main difference with respect to least squares is that it can be applied in real time, which increases its usefulness in a real world scenario. They generally work by estimating the probability density function (pdf) of the state distribution. In the case of linear systems, this is a Gaussian distribution, which is fully determined by its mean vector and covariance matrix. The mean and covariance are updated analytically within the KF algorithm. In practice, the algorithm

*Ph.D. Student, System & Industrial Engineering Department, University of Arizona, 1127 E. James E. Rogers Way, Tucson, AZ 85721

[†]Professor, System & Industrial Engineering Department, University of Arizona, 1127 E. James E. Rogers Way, Tucson, AZ 85721

is often used for non-linear systems by linearizing the system's process and measurement models. Different ways of linearizing the models lead to different filters, namely Extended Kalman Filter (EKF), the Iterated EKF (IEKF)⁵ and the Linear Regression Kalman Filter (LRKF).⁶

The goal of this work is to use neural network to solve the OD problem. Examples of machine learning and neural networks being applied to OD problems are present but the field is relatively new. Furfaro et al.⁷ demonstrated that it is possible to create an accurate inverse mapping between angle measurements and orbital parameters using Extreme Learning Machines. Sharma⁸ presented an algorithm for orbit determination and magnetometer interference cancellation for cub-sats, demonstrating robustness in a variety of scenarios (e.g. Doppler-only, position based, lunar orbit and low Earth orbit). Lee et al.⁹ presented a new interesting way to simulate range rate and slant range observations using machine learning. Peng et al.¹⁰ also demonstrated the viability of machine learning, specifically Artificial Neural Networks (ANNs), for orbit prediction accuracy improvement.

The goal of this work is to build upon the foundation of ML applied to the OD problem using the recently developed Physics-Informed Extreme Learning Machines (PIELM) to regularize the learning algorithm and improve dynamical soundness of the estimated solution. Standard PIELM are particular neural networks, developed for solving forward and inverse problems governed by parametric Differential Equations (DEs). The training of these networks, as in any Physics Informed method, is regulated by the physics of the problem. In particular, the latent solution of the parametric DE is approximated via Neural Networks (NNs); the DE that models the physics of the problem, is then embedded, in its implicit form, into the NN loss function. Thus, the DE drives the training of the network acting as regulator that penalizes the network's training when the physics of the problem is violated. Usually these NN are then trained via gradient-based methods. In case the method employs Extreme Learning Machines (ELM), as in this case, the training can be carried out in one single step using minimum norm least squares. With this work we harness the power and ease of implementation of neural network to solve a set of OD problems based on least squares. This leads to an algorithm capable of performance similar to batch least squares with some advantages. Specifically, the algorithm is capable of estimating the complete state evolution without any integration step and without the need to provide an initial guess, producing in the meantime more accurate results.

The rest of the paper is organized as follows: Section II introduces the concepts of orbit determination and physics informed neural networks as building blocks of our method, Section III describes the Physics Informed Least Squares (PILS) in details, Section IV shows the numerical results of the method and compares it with classical least squares, and finally Section V contains the conclusion statement.

THEORETICAL CONCEPTS

In this section the building blocks of the physics informed orbit determination techniques are introduced: the classical orbit determination problem and the physics informed neural networks in the form of PIELM.

The orbit determination problem

The orbit determination problem is normally cast as an estimation problem where a set of measurements of the same or a collections of quantities related to the problem are used to estimate the

state of the system. The major techniques in this field are Least Squares (and Batch Least Squares) estimate and Kalman Filters. We focus on the first group in this case as foundation concept to create our OD framework. According to least squares, suppose there is a set, or batch, of measured values $\tilde{y}_j$, taken at discrete time instants t_j :

$$\{\tilde{y}_1, t_1; \tilde{y}_2, t_2; \dots; \tilde{y}_m, t_m\} \quad (1)$$

and a mathematical model correlating the measurement with some constants x_i :

$$y(t) = \sum_{i=1}^n x_i h_i(t) \quad (2)$$

where $h_i(t)$ are a set of basis functions. The x_i are a set of constants whose numerical values are unknown. In classical least squares, we seek a set of estimates, denoted by $\{\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n\}$, which can be used to predict $y(t)$. Measurement errors, however, can arise between the “true” value $y(t)$ and the predicted (estimated) value $\hat{y}(t)$. We consider the measurement error to be some unknown random process. We assume that the measurement $\tilde{y}_j$ and the estimated output $\hat{y}_j$ are modelled by the equations:

$$\tilde{y}_j = \sum_{i=1}^n x_i h_i(t_j) + \nu_j \quad (3)$$

$$\hat{y}_j = \sum_{i=1}^n \hat{x}_i h_i(t_j) \quad (4)$$

where ν_j is the measurement error. We can then define the following identity:

$$\tilde{y}_j = \sum_{i=1}^n \hat{x}_i h_i(t_j) + e_j \quad (5)$$

where e_j is the residual error between the measured and predicted measurement defined as:

$$e_j = \tilde{y}_j - \hat{y}_j \quad (6)$$

Gauss’s principle of least squares selects, as an optimum choice for the unknown parameters, the particular $\hat{x}$ that minimizes the sum square of the residual errors, given by

$$J = \frac{1}{2} e^T e \quad (7)$$

In case of classical least squares, this is expressed using the above definitions:

$$J = \frac{1}{2} (\tilde{\mathbf{y}}^T \tilde{\mathbf{y}} - 2\tilde{\mathbf{y}}^T \mathbf{H} \hat{\mathbf{x}} + \hat{\mathbf{x}}^T \mathbf{H}^T \mathbf{H} \hat{\mathbf{x}}) \quad (8)$$

We can then take the gradient of J to obtain the necessary condition of optimality:

$$\nabla_{\hat{\mathbf{x}}} J = \mathbf{H}^T \mathbf{H} \hat{\mathbf{x}} - \mathbf{H}^T \tilde{\mathbf{y}} = 0 \quad (9)$$

Which can then be solved for $\hat{x}$:

$$\hat{\mathbf{x}} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \tilde{\mathbf{y}} \quad (10)$$

In this case we used the weighted least squares which is based on the method just described. It is obtained by considering the following objective function:¹

$$J = \frac{1}{2} \mathbf{e}^T \mathbf{W} \mathbf{e} \quad (11)$$

where $\mathbf{W}$ is a diagonal weighting matrix. The solution is then derived in the same manner as before:

$$\nabla_{\hat{\mathbf{x}}} J = \mathbf{H}^T \mathbf{W} \mathbf{H} \hat{\mathbf{x}} - \mathbf{H}^T \mathbf{W} \tilde{\mathbf{y}} = 0 \quad (12)$$

and solving for $\hat{\mathbf{x}}$:

$$\hat{\mathbf{x}} = (\mathbf{H}^T \mathbf{W} \mathbf{H})^{-1} \mathbf{H}^T \mathbf{W} \tilde{\mathbf{y}} \quad (13)$$

The weight matrix can in theory be any diagonal matrix but it is known that by choosing it as the inverse of the observation error covariance matrix $\mathbf{R}^{-1}$, the least squares solution gives the minimum variance estimator.

Range and range rate OD problem We develop here the classical least squares solution to the initial state estimation given a set of range and range rate observations from 3 separate ground stations. The problem is the unperturbed 2-body problem whose equations of motion are:

$$\dot{\mathbf{r}} = \mathbf{v} \quad (14)$$

$$\dot{\mathbf{v}} = -\frac{\mu}{r^3} \mathbf{r} \quad (15)$$

In this case $\mathbf{x} = [\mathbf{r}, \mathbf{v}]$. We consider, the ground stations to be in positions $\mathbf{r}_{s1}, \mathbf{r}_{s2}, \mathbf{r}_{s3}$ in the ECEF* frame and we consider them known. The non-linear nature of the problem requires the usage of a non-linear least squares estimation.² In this case the real and estimated measurements are expressed as:

$$\tilde{\mathbf{y}} = f(\mathbf{x}) + \boldsymbol{\nu} \quad (16)$$

$$\hat{\mathbf{y}} = f(\hat{\mathbf{x}}) \quad (17)$$

where $f(\cdot)$ is a non-linear function of the real or estimated state. The error is:

$$\mathbf{e} = \tilde{\mathbf{y}} - \hat{\mathbf{y}} \quad (18)$$

The objective function is then:

$$J = \frac{1}{2} \mathbf{e}^T \mathbf{W} \mathbf{e} \quad (19)$$

This is optimized by linearizing the model around a reference trajectory and solving for the variations as opposed to the real state. This works by initializing the vector $\mathbf{x}_0$ with an initial guess. Given this initial guess, generate the reference trajectory by integrating the equations of motion and simultaneously integrating the STM using $\dot{\Phi} = \mathbf{A}(t)\Phi$, with $\mathbf{A}(t)$ being the linearized system matrix and initial conditions $\Phi(0, 0) = \mathbf{I}$. The estimated state can be expressed as:

$$\hat{\mathbf{x}} = \mathbf{x}_c + \delta \mathbf{x} \quad (20)$$

The observation model is:

$$f(\hat{\mathbf{x}}) \approx f(\mathbf{x}_c) + \mathbf{H} \delta \mathbf{x} \quad (21)$$

*Earth-centered, Earth-fixed

Where $\mathbf{H} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}_c}$ is the jacobian calculated over the reference trajectory. The observations variation is:

$$\delta \mathbf{y} = \tilde{\mathbf{y}} - f(\hat{\mathbf{x}}) \approx \tilde{\mathbf{y}} - f(\mathbf{x}_c) - \mathbf{H} \delta \mathbf{x} = \delta \mathbf{y}_c - \mathbf{H} \delta \mathbf{x} \quad (22)$$

where $f(\mathbf{x}_c)$ are the observations computed on the reference trajectory. The objective is then:

$$J = \frac{1}{2} \delta \mathbf{y}^T \mathbf{W} \delta \mathbf{y} = (\delta \mathbf{y}_c - \mathbf{H} \delta \mathbf{x})^T \mathbf{W} (\delta \mathbf{y}_c - \mathbf{H} \delta \mathbf{x}) \quad (23)$$

Which can be optimized the same way as 8. So the solution for the initial state variation is:

$$\delta \hat{\mathbf{x}} = (\mathbf{H}^T \mathbf{W} \mathbf{H})^{-1} \mathbf{H}^T \delta \mathbf{y} \quad (24)$$

The initial state estimate is then updated:

$$\hat{\mathbf{x}}_0^{k+1} = \hat{\mathbf{x}}_0^k + \delta \hat{\mathbf{x}} \quad (25)$$

and a new reference trajectory computed. This is iterated until the convergence criterion $\|\hat{\mathbf{x}}_0^{k+1} - \hat{\mathbf{x}}_0^k\| \leq \epsilon$ is met.

Physics Informed Extreme Learning Machines (PIELM)

To properly define PIELM¹¹ it is important to introduce Physics Informed Neural Networks (PINN), as PIELM are a particular subset of them and share the basic idea. PINN where first introduced by Raissi et al.¹² as a method for approximating solutions to general non-linear PDEs as well as some inverse problems. Unlike finite elements and mesh-based method in general, PINN uses a deep neural network (DNN) as basis function to approximate the solution. The PDE and its boundary conditions are sampled on locations within and at the boundaries of the domain respectively. The output of the DNN at those locations is used to compute the residuals of the PDE on the internal points and on the boundaries. The loss is then expressed as the summation of the errors on the known boundary conditions and the residuals based on the physical relationships of the problem. The weights and biases of the neural network are then learned by minimizing the loss using a gradient-based optimization method. It should be noted that, although they were introduced for solving PDEs, they can also be employed to solve ODEs and systems of ODEs where the only input to the network is time and the boundary conditions are not defined. This has shown to work well in a variety of continuous and discrete time models.¹² In mathematical terms, the basics of PINN can be described by considering a general problem in the form:

$$u_t + \mathcal{N}[u] = 0, \quad x \in \Omega, \quad t \in [0, T] \quad (26)$$

where u is the latent solution and $\mathcal{N}[\cdot]$ is any nonlinear differential operator. We then define $f(t, x)$ as the left-hand side of equation 26:

$$f(t, x) = u_t + \mathcal{N}[u] \quad (27)$$

The main idea of PINN is to approximate the solution $u(t, x)$ with a neural network. The neural network then becomes a physics informed neural network by training its weights and biases by minimizing the mean squared error loss:

$$MSE = MSE_u + MSE_f \quad (28)$$

where

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2 \quad (29)$$

and

$$MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2 \quad (30)$$

$\{t_u^i, x_u^i, u^i\}_{i=1}^{N_u}$ denote the initial and boundary training data on $u(t, x)$ and $\{t_f^i, x_f^i\}_{i=1}^{N_f}$ denote the collocation points within the domain for $f(t, x)$. This loss ensures that the solution respects both the dynamics and the initial/boundary conditions.

PIELM is an hybrid neural network-based method that integrates the concepts of PINN explained above and the power of Extreme Learning Machines (ELM).¹³ ELM are a particular kind of single layer feed-forward networks in which the input weights and biases are randomly chosen and constant through the learning process while the output weights are the only trained parameters. This allows the training to be carried out in a single step which dramatically decreases computational time. ELM also have been shown to have outstanding representation power and better generalization than classical gradient-based learning procedures.¹⁴ In mathematical terms, considering a single layer feed forward network with n neurons, $\mathbf{x}$ is the input state and y is the output, an ELM is a mapping between input and output through a series of non-linear basis functions called activation functions σ and a set of parameters called input weights ($\mathbf{w}$), biases ($\mathbf{b}$) and output weights (β). The output of a single neuron is $\sigma(\mathbf{w}_j^T \mathbf{x} + b_j)$. For a given dataset $\{\mathbf{x}_k, y_k\}_{k=1}^N$ with N samples, the output is given by:

$$\mathbf{y} = \mathbf{H}(\mathbf{x})\beta \quad (31)$$

where

$$\mathbf{H} = [\mathbf{h}(\mathbf{x}_1), \mathbf{h}(\mathbf{x}_2), \dots, \mathbf{h}(\mathbf{x}_T)]^T \quad (32)$$

is the output of the hidden layer, with

$$\mathbf{h}(\mathbf{x}_k) = [\sigma(\mathbf{w}_1^T \mathbf{x}_k + b_1), \sigma(\mathbf{w}_2^T \mathbf{x}_k + b_2), \dots, \sigma(\mathbf{w}_n^T \mathbf{x}_k + b_n)] \quad (33)$$

Training is achieved by minimizing the error between the predicted output and the real output $\mathbf{T}$ $\epsilon = \mathbf{y} - \mathbf{T}$:

$$J = \frac{1}{2N} \epsilon^T \epsilon \quad (34)$$

The solution is obtained analytically as:

$$\beta = \tilde{\mathbf{H}}\mathbf{T} \quad (35)$$

where $\tilde{\mathbf{H}}$ is the pseudoinverse of matrix $\mathbf{H}$ which is in general not squared. Figure 1 shows a representation of an ELM. So PIELM reduces to choosing an ELM as a parametric representation of the state of the PDE/ODE and learn its parameters by optimizing a physics-based loss function i.e. the summation of PDE and BCs least-squares cost as explained above.

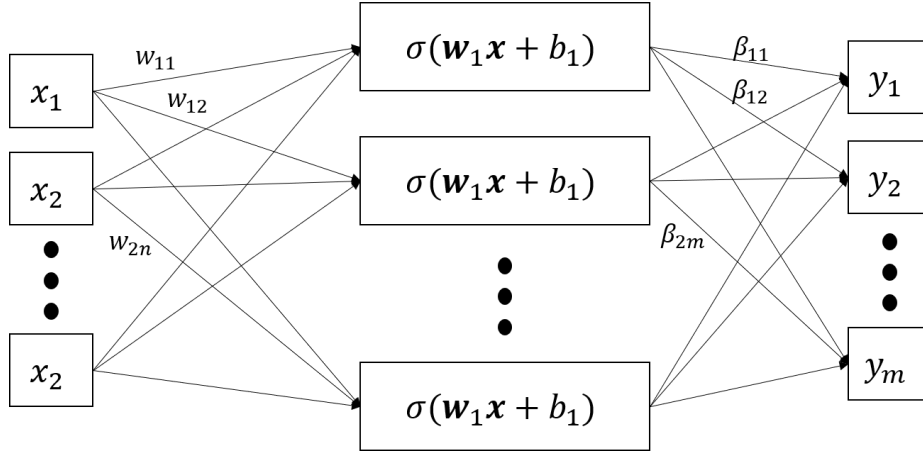


Figure 1. ELM architecture

PHYSICS INFORMED LEAST SQUARES (PILS)

The way we use PIELM to solve the OD problem is based on approximating the state of the system with an ELM as a function of the epoch t . In this case we want to estimate the whole state of the system $\mathbf{x} = [\mathbf{r}, \mathbf{v}]$, comprising of position and velocity. Due to the simplicity of a single layer network, we can achieve that by creating a network whose output is the 3D position $\mathbf{r}$ and obtain the velocity by analytically differentiating the output. This being said, the position of the satellite is expressed as:

$$\mathbf{r} = ELM(t) \quad (36)$$

We initialize the input weights $\mathbf{w}$ and biases $\mathbf{b}$ as random vectors between -1 and 1 of length n , where n is the number of neurons. The output weights $\boldsymbol{\xi}$ are also initialized as a random vector of length $3n$ between 0 and 1.

The matrix $\mathbf{F} = \mathbf{H}$, is obtained, along with its derivatives, using a hyperbolic tangent activation function

$$\mathbf{F} = \mathbf{H} \quad (37)$$

$$\dot{\mathbf{F}} = \frac{d\mathbf{H}}{dt} \quad (38)$$

$$\ddot{\mathbf{F}} = \frac{d^2\mathbf{H}}{dt^2} \quad (39)$$

$$(40)$$

The change of name is introduced here to differentiate them from the ELM formalization. The output of the net (i.e. the position of the satellite) and its derivatives are then:

$$\mathbf{r} = [r_1, r_2, r_3] = [\mathbf{F}^T \boldsymbol{\xi}_1, \mathbf{F}^T \boldsymbol{\xi}_2, \mathbf{F}^T \boldsymbol{\xi}_3] \quad (41)$$

$$\dot{\mathbf{r}} = [\dot{r}_1, \dot{r}_2, \dot{r}_3] = [\dot{\mathbf{F}}^T \boldsymbol{\xi}_1, \dot{\mathbf{F}}^T \boldsymbol{\xi}_2, \dot{\mathbf{F}}^T \boldsymbol{\xi}_3] \quad (42)$$

$$\ddot{\mathbf{r}} = [\ddot{r}_1, \ddot{r}_2, \ddot{r}_3] = [\ddot{\mathbf{F}}^T \boldsymbol{\xi}_1, \ddot{\mathbf{F}}^T \boldsymbol{\xi}_2, \ddot{\mathbf{F}}^T \boldsymbol{\xi}_3] \quad (43)$$

$$(44)$$

Where $\xi = [\xi_1, \xi_2, \xi_3]$ has been split into three parts associated to the three components of the position. We then use this mapping to approximate the measurement using the same measurement model derived for the classical least square estimation described above. Specifically, we use range ρ and range rate $\dot{\rho}$ from three distinct ground stations. By using the mapping, these components take the form

$$\begin{aligned} \rho_{NN_i} &= \sqrt{(r_1 - x_{s_i} \cos \theta + y_{s_i} \sin \theta)^2 + (r_2 - x_{s_i} \sin \theta - y_{s_i} \cos \theta)^2 + (r_3 - z_{s_i})^2} \\ \dot{\rho}_{NN_i} &= [(r_1 - x_{s_i} \cos \theta + y_{s_i} \sin \theta)(\dot{r}_1 + x_{s_i} \sin \theta \dot{\theta} + y_{s_i} \cos \theta \dot{\theta}) + \\ &\quad (r_2 - x_{s_i} \sin \theta - y_{s_i} \cos \theta)(\dot{r}_2 - x_{s_i} \cos \theta \dot{\theta} + y_{s_i} \sin \theta \dot{\theta}) + \\ &\quad (r_3 - z_{s_i})\dot{r}_3] / \rho_{NN_i} \end{aligned} \quad (45)$$

where i refers to the i -th ground station and the subscript NN is to differentiate it from the real observations ρ_i and $\dot{\rho}_i$. θ and $\dot{\theta}$ are the angular rotation and rotation rate of the ECEF frame with respect to the Earth Centered Inertial (ECI) frame. The residual is then:

$$R = [R_{f_1}, R_{f_2}, R_{f_3}, R_{\rho_1}, R_{\rho_2}, R_{\rho_3}, R_{\dot{\rho}_1}, R_{\dot{\rho}_2}, R_{\dot{\rho}_3}] \quad (46)$$

where

$$R_{f_i} = \ddot{r}_i + \frac{\mu}{r^3} r_i \quad (47)$$

$$R_{\rho_i} = \rho_i - \rho_{NN_i} \quad (48)$$

$$R_{\dot{\rho}_i} = \dot{\rho}_i - \dot{\rho}_{NN_i} \quad (49)$$

$$(50)$$

These residuals need to be minimized to solve the OD problem. The parallelism with PINN is evident as the terms R_{ρ_i} and $R_{\dot{\rho}_i}$ ensure that the solution is close to the observations (which act as constraint in this case), and the term R_{f_i} ensure that the solution respects the physics of the problem. The residuals are then analytically differentiated with respect to the parameters to learn ξ . The resulting derivatives are arranged in a matrix representing the jacobian of the method:

$$J = \frac{dR}{d\xi} \quad (51)$$

Being the problem non-linear, we cannot solve directly for ξ . Instead the solution is refined using an iterative process. The solution for the weights variation $\delta\xi$ is obtained by solving the minimum norm least squares problem:

$$\min_{\delta\xi} \|J\delta\xi - R\| \quad (52)$$

which also minimizes $\|\delta\xi\|$. The weight vector is then updated:

$$\xi^{k+1} = \xi^k + \delta\xi \quad (53)$$

until the convergence criterion $\|R\| < \epsilon$ is met.

Table 1. Test Orbits

	a (km)	e (deg)	i (deg)	ω (deg)	Ω (deg)	θ_0 (deg)
LEO	9000	0.1	60	0	45	0
GEO	42164	0	0	0	0	100
Molniya	26600	0.74	63.4	270	0	190

Table 2. Ground Stations ECEF coordinates

	x (km)	y (km)	z (km)
Pacific Station	-5127.51	-3794.16	0.0
Pirinclik	3860.90	3238.50	3898.10
Thule	549.50	-1380.87	6182.20

NUMERICAL RESULTS

The proposed method is tested and compared to classical least squares solution. It is tested on 3 orbit determination scenarios. First we consider a low earth orbit (LEO), sampling 300 measurements over a period of 500 seconds. Then we consider a geostationary orbit (GEO) sampling 300 measurements over a 10000 seconds period. Finally we consider a Molniya orbit, sampling 300 samples over 5000 seconds. The orbital parameters for all the orbits are given in Table 1. The measurements consist of range and range rate values from three ground stations whose coordinates in the ECEF frame are reported in Table 2. It should be noted that the ground stations reported in the table are chosen purely for testing purposes and are not related to any specific mission requirements. The method is independent of the ground stations selection. The observations are created by adding a gaussian noise to the real range and range rate. The standard deviations of the errors on range and range rate are 1 m and 0.1 m/s respectively. The number of neurons n is 10 in all cases.

Tables 3, 4, 5 shows the true initial state, the estimated initial state and the error, both with classical least squares (LS) and PILS. The results show very comparable performance with respect to classical least squares and in many cases the accuracy on the initial state estimation is improved. Figures 2,3,4 show the orbits and the part of trajectory considered for the estimation process, along with the real and estimated states and the error between them. It is interesting to note that since PILS estimates all the states at once, the errors do not show particular increasing trends, which might happen when estimating the states by propagating the estimated initial state. Table 6 shows the RMSE between the states of the whole arc estimated using PILS and the integrated states starting from the initial state estimated using LS. This shows PILS outperforms classical LS in most of the cases, struggling to beat LS only for some of the velocity components. It should be noted that the parameters of the PILS (i.e. number of neurons) have not been optimized so there might be room for improvement.

CONCLUSION

The proposed method combines the power of machine learning and least squares orbit determination techniques to create a new orbit determination method named Physics Informed Least Squares (PILS). It is based on PIELM where an ELM is used to approximate the latent solution of an ODE or PDE. This works by penalizing the loss function of the optimization method used to obtain the

Table 3. LEO: initial state estimate (all quantities in km and km/s)

	real	estimated LS	estimated PILS	error LS	error PILS
x	5727.5649	5727.5669	5727.5663	2.07×10^{-3}	1.38×10^{-3}
y	5727.5649	5727.5626	5727.5635	-2.25×10^{-3}	-1.40×10^{-3}
z	0.0	0.0016	-0.0006	1.69×10^{-3}	-6.18×10^{-4}
$\dot{x}$	-2.6012	-2.6012	-2.6011	1.25×10^{-5}	-8.02×10^{-6}
$\dot{y}$	2.6012	2.6012	2.6011	7.29×10^{-6}	8.87×10^{-6}
$\dot{z}$	6.3716	6.3716	6.3715	3.41×10^{-6}	1.25×10^{-6}

Table 4. GEO: initial state estimate (all quantities in km and km/s)

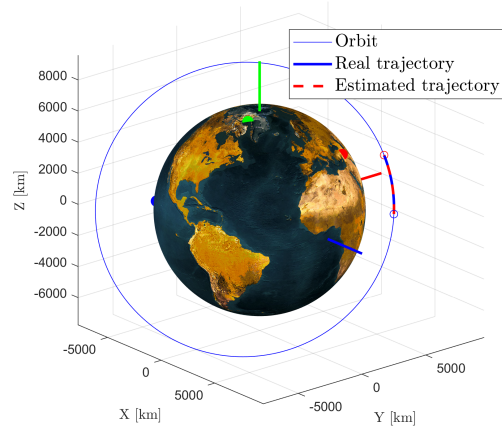
	real	estimated LS	estimated PILS	error LS	error PILS
x	-7321.7017	-7321.6989	-0.7321.7033	2.77×10^{-3}	-1.61×10^{-3}
y	41523.4341	41523.4349	41523.4341	8.28×10^{-4}	6.59×10^{-5}
z	0	3.9437×10^{-4}	1.7603×10^{-4}	3.94×10^{-4}	1.76×10^{-3}
$\dot{x}$	-3.0279	-3.0279	-3.0279	-2.52×10^{-8}	9.07×10^{-8}
$\dot{y}$	-0.5339	-0.5339	-0.5339	2.17×10^{-7}	-3.64×10^{-8}
$\dot{z}$	0	2.56×10^{-9}	-4.3728×10^{-7}	2.56×10^{-9}	-4.37×10^{-7}

Table 5. Molniya: initial state estimate (all quantities in km and km/s)

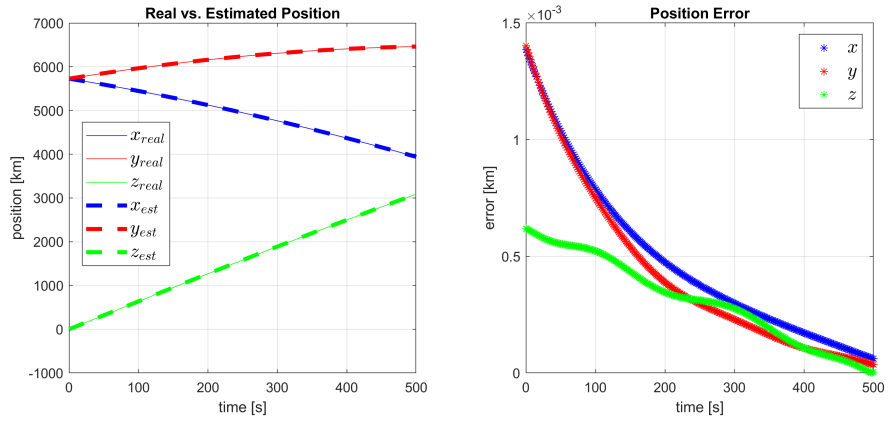
	real	estimated LS	estimated PILS	error LS	error PILS
x	-7704.0147	-7703.9989	-7704.0144	1.57×10^{-2}	3.23×10^{-4}
y	19563.3282	19563.3127	19563.3287	-1.55×10^{-2}	5.12×10^{-4}
z	39067.0638	39067.0739	39067.0637	1.01×10^{-2}	-9.14×10^{-5}
$\dot{x}$	-1.4089	-1.4089	-1.4089	1.90×10^{-7}	-1.56×10^{-7}
$\dot{y}$	-0.4474	-0.4474	-0.4474	2.01×10^{-6}	1.01×10^{-6}
$\dot{z}$	-0.8936	-0.8936	-0.8936	5.29×10^{-7}	7.44×10^{-9}

Table 6. RMSE for all cases (all quantities in km and km/s)

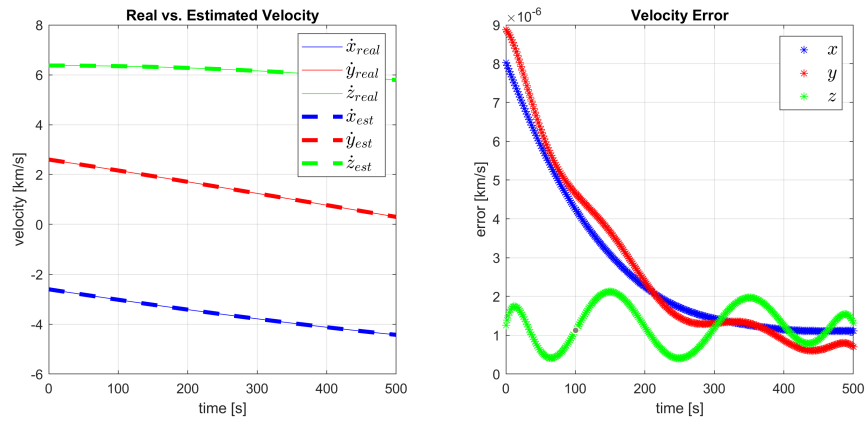
	LEO LS	LEO PILS	GEO LS	GEO PILS	Molniya LS	Molniya PILS
x	5.52×10^{-3}	5.92×10^{-4}	2.40×10^{-3}	7.75×10^{-4}	1.58×10^{-2}	4.05×10^{-4}
y	1.26×10^{-3}	5.59×10^{-4}	1.97×10^{-3}	3.31×10^{-4}	1.06×10^{-2}	1.05×10^{-3}
z	2.54×10^{-3}	3.60×10^{-4}	3.73×10^{-4}	1.49×10^{-3}	1.13×10^{-2}	5.81×10^{-4}
$\dot{x}$	1.25×10^{-5}	3.27×10^{-6}	1.14×10^{-7}	4.29×10^{-7}	1.37×10^{-7}	4.23×10^{-7}
$\dot{y}$	8.47×10^{-6}	3.54×10^{-6}	2.02×10^{-7}	1.13×10^{-7}	2.17×10^{-6}	1.31×10^{-6}
$\dot{z}$	3.24×10^{-6}	1.35×10^{-6}	9.67×10^{-9}	4.64×10^{-7}	4.14×10^{-7}	6.06×10^{-7}



(a) Trajectory

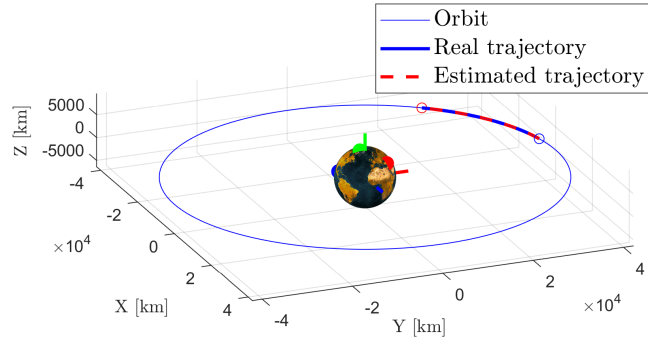


(b) Position

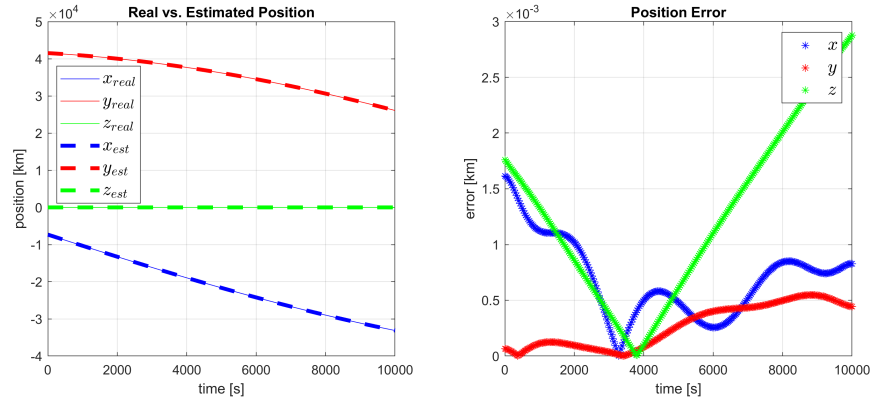


(c) Velocity

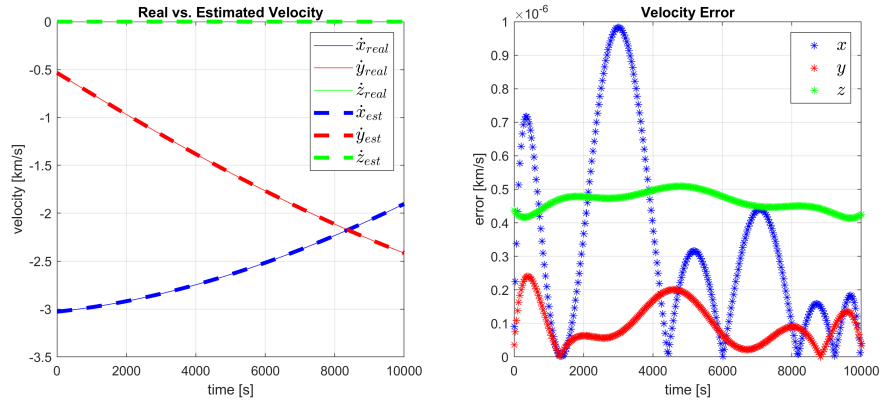
Figure 2. LEO results



(a) Trajectory

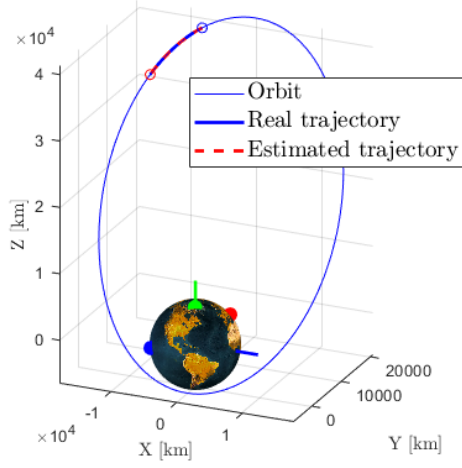


(b) Position

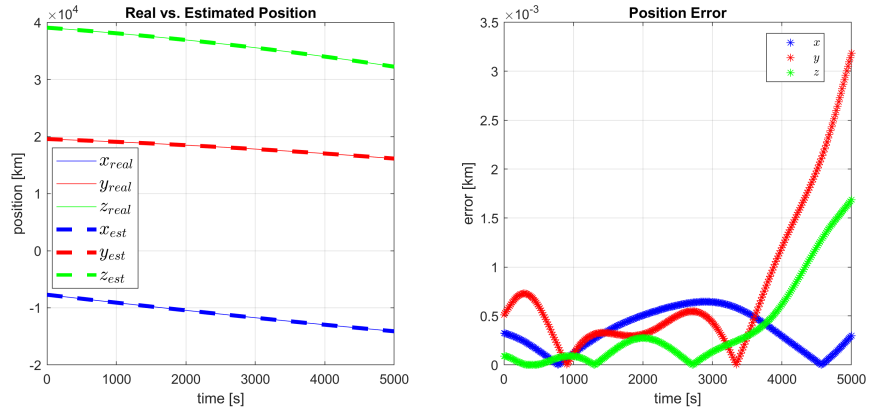


(c) Velocity

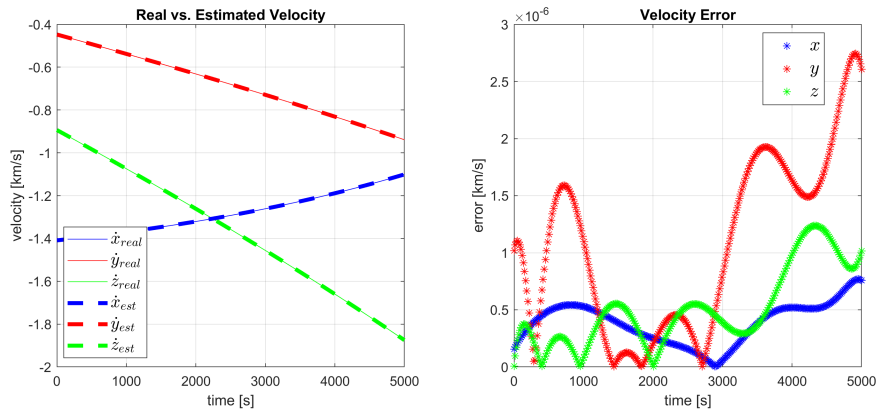
Figure 3. GEO results



(a) Trajectory



(b) Position



(c) Velocity

Figure 4. Molniya results

parameters of the network with the dynamical system. In this case the loss is composed by a the error between the computed range and range rate measurement, given the position output of the net and its derivatives, and the noisy observations coming from three ground stations. This loss is then augmented with the dynamics residual based on the equations of motion, hence the physics informed definition. The method performs exceptionally well in three different test scenarios: a LEO, a GEO and a Molniya orbit. In particular, the performance is comparable or superior to classical batch least squares when estimating the initial state of the arc. Moreover, since PILS estimates the whole arc as opposed to LS, the RMSE using PILS on the whole arc is lower for almost all states in all cases, when compared to the integrated arc starting from the initial state estimated using LS. Also, the particular structure of PILS allows it to provide an accurate estimate without needing an initial state guess, which might be cumbersome to obtain in a real world scenario. Overall, with this paper we prove that neural networks are a powerful tool that can be employed for orbit determination, which manages to perform on par or better than existing least squares techniques, even without parameter tuning. Possible improvement might be obtained by optimizing the network parameters depending on the task at hand.

REFERENCES

- [1] J. L. Crassidis and J. L. Junkins, *Optimal estimation of dynamic systems*. CRC press, 2011.
- [2] B. Schutz, B. Tapley, and G. H. Born, *Statistical orbit determination*. Elsevier, 2004.
- [3] R. G. Brown and P. Y. Hwang, *Introduction to random signals and applied Kalman filtering: with MATLAB exercises*, Vol. 4. John Wiley & Sons New York, NY, USA, 2012.
- [4] T. Lefebvre*, H. Bruyninckx, and J. De Schutter, “Kalman filters for non-linear systems: a comparison of performance,” *International journal of Control*, Vol. 77, No. 7, 2004, pp. 639–653.
- [5] L. Fan and Y. Wehbe, “Extended Kalman filtering based real-time dynamic state and parameter estimation using PMU data,” *Electric Power Systems Research*, Vol. 103, 2013, pp. 168–177.
- [6] T. Lefebvre, H. Bruyninckx, and J. De Schutter, “A The Linear Regression Kalman Filter,” *Nonlinear Kalman Filtering for Force-Controlled Robot Tasks*, pp. 205–210, Springer, 2005.
- [7] R. Furfaro, R. Linares, M. K. Jah, and D. Gaylor, “Mapping sensors measurements to the resident space objects behavior energy and state parameters space via extreme learning machines,” *International Astronautical Congress*, 2016.
- [8] S. Sharma, *Machine Learning Applications in Spacecraft State and Environment Estimation*. PhD thesis, 2018.
- [9] B.-s. Lee, W.-G. Kim, J. Lee, and Y. Hwang, “Machine Learning Approach to Initial Orbit Determination of Unknown LEO Satellites,” *2018 SpaceOps Conference*, 2018, p. 2566.
- [10] H. Peng and X. Bai, “Artificial neural network–based machine learning approach to improve orbit prediction accuracy,” *Journal of Spacecraft and Rockets*, Vol. 55, No. 5, 2018, pp. 1248–1260.
- [11] V. Dwivedi and B. Srinivasan, “Physics Informed Extreme Learning Machine (PIELM)—A rapid method for the numerical solution of partial differential equations,” *Neurocomputing*, Vol. 391, 2020, pp. 96–118.
- [12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, Vol. 378, 2019, pp. 686–707.
- [13] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, “Extreme learning machine: theory and applications,” *Neurocomputing*, Vol. 70, No. 1-3, 2006, pp. 489–501.
- [14] G.-B. Huang, L. Chen, C. K. Siew, *et al.*, “Universal approximation using incremental constructive feedforward networks with random hidden nodes,” *IEEE Trans. Neural Networks*, Vol. 17, No. 4, 2006, pp. 879–892.